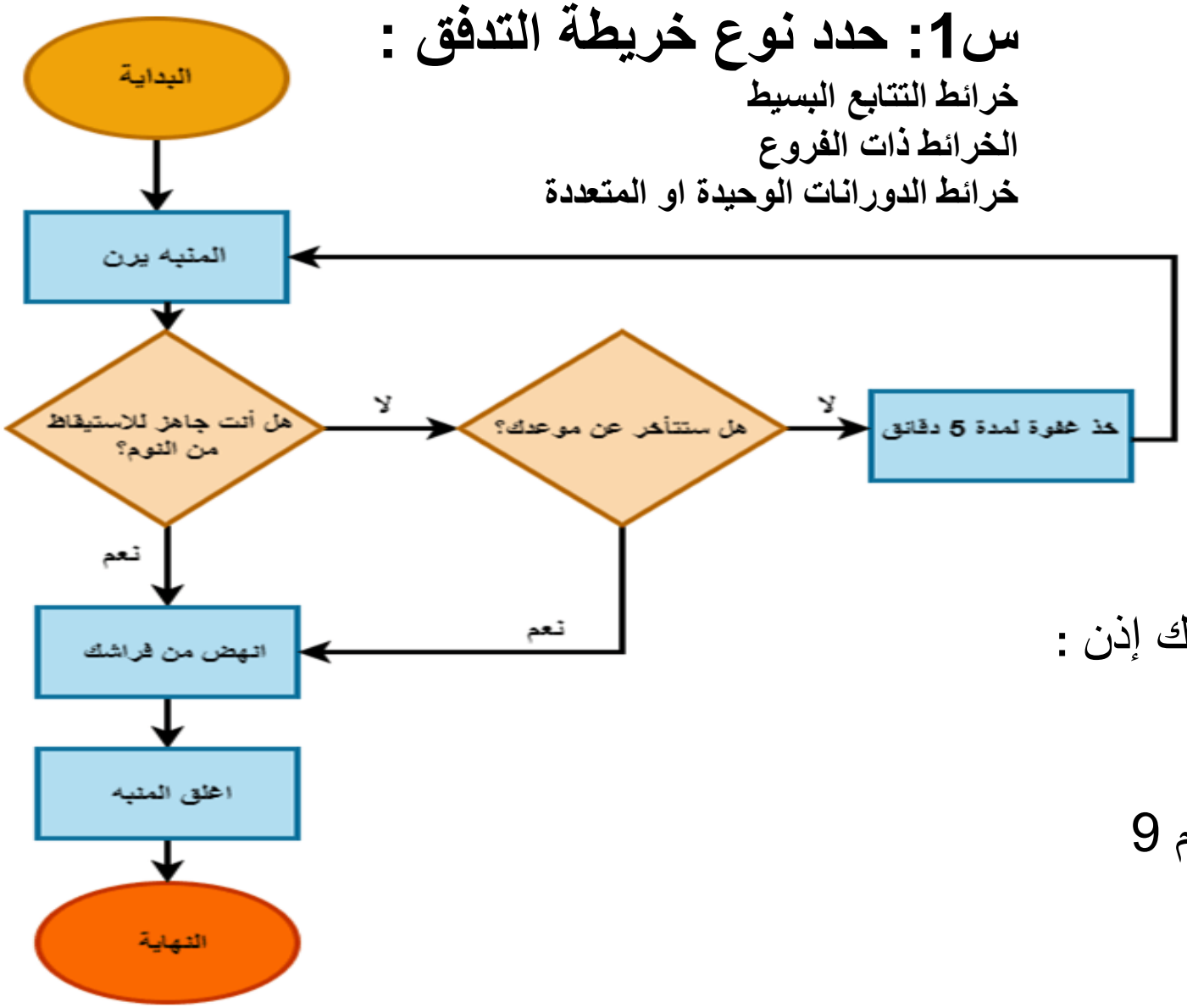
تصميم برامج الحاسب الالى
محاضرة 2

د/الشيماء عبدالرؤوف

اختبر نفسك

س1: حدد نوع خريطة التدفق :

خرائط التتابع البسيط
الخرائط ذات الفروع
خرائط الدورانات الوحيدة او المتعددة



خطوات الحـــــــــل

1 – بداية
2 – افتح المنبه
3 – إذا كنت جاهزًا للاستيقاظ إذن :
3 – 1 – انهض من فراشك
3 – 2 – أغلق المنبه
3 – 3 – اذهب إلى الخطوة رقم 9
4 – غير ذلك :
4 – 1 – إذا كنت ستتأخر عن موعدك إذن :
4 – 1 – 1 – انهض من فراشك
4 – 1 – 2 – أغلق المنبه
4 – 1 – 3 – اذهب إلى الخطوة رقم 9
5 – غير ذلك :
5 – 1 – خذ غفوة لمدة 5 دقائق
5 – 2 – اذهب إلى الخطوة رقم 2
9 – نهاية

كيف يعمل برنامج الحاسب الالى

- الكمبيوترات تفهم فقط **لغة الآلة**، وهي لغة منخفضة المستوى تتكون من **أرقام ثنائية** (0 و 1).
  لتقريب الفجوة بين البشر ولغة الآلة، نستخدم **لغات البرمجة**.
- لكي نبرمج الكمبيوتر، نحتاج إلى كتابة التعليمات بلغة خاصة نسميها **لغة برمجة**.
  نسمي التعليمات التي نرسلها إلى الكمبيوتر **كود**. وكل سطر من التعليمات نسميه **سطر كود**.
- عندما نكتب الكود، نحن نبرمج الكمبيوتر ليقوم بشيء معين. ولهذا السبب، نسمي الكود الذي نكتبه أيضًا **برنامج حاسب** (أو برنامج).

ما المقصود بالبرمجــة ؟

- **برمجة الحاسوب** : هى عملية تزويد الكمبيوتر بالأوامر اللازمة لتنفيذ مهمة معينة بطريقة معينة .
- تنقسم لغات البرمجه الى :

❑ **لغات منخفضة المستوى:**

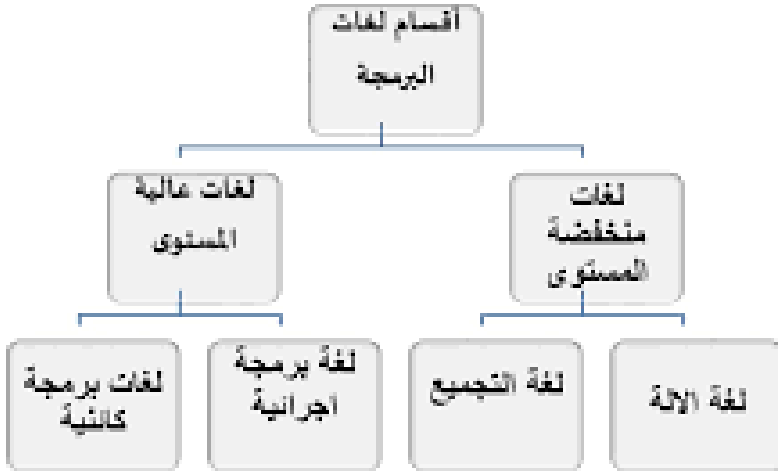
1- لغة الالة:هي اللغة الوحيدة التي يفهمها الحاسب ويستطيع التعامل معها.

قد تختلف لغة الآلة من حاسب إلى آخر، ولذلك نجد أن نفس البرنامج الذي يعمل على حاسب معين قد لا يعمل على حاسب آخر يختلف عنه في المكونات المادية.

2-لغة التجميع :هي لغة تستخدم اختصارات معبرة من اللغة الإنجليزية لتعبر بها عن العمليات الأولية التي يقوم بها الحاسب

ولكنها تعتمد على نوع الحاسب كما انها صعبة التعلم

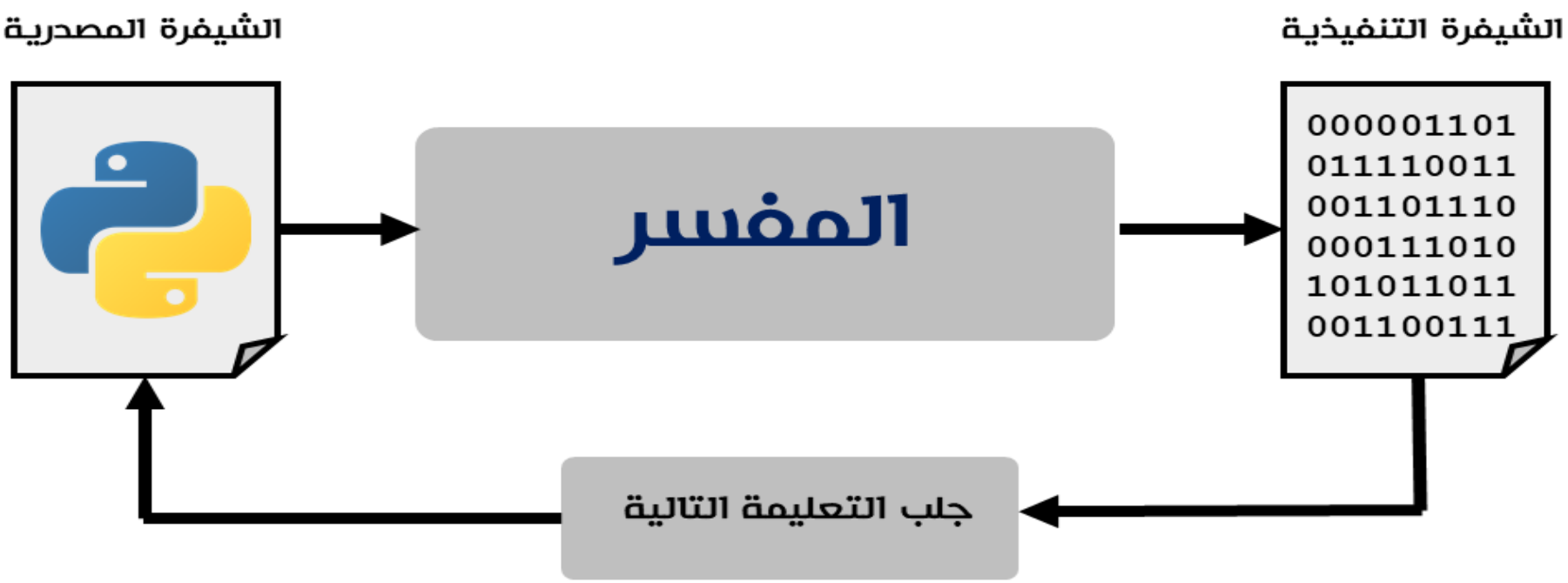
❑**لغات عالية المستوى**: تستخدم كلمات اوضح من اللغة الانجليزية اسهل فى الاستخدام والصيانه والتصحيح



تطور لغات البرمجة

لغة الالة	لغة التجميع	لغات عالية المستوى
0110 0011 0010 0001	LDA X	$D = X + Y + Z$
0100 0011 0010 0010	ADA Y	
0100 0011 0010 0011	ADA Z	
0111 0011 0010 0100	STA D	

وجميع اللغات عالية المستوى يجب تحويل برامجها إلى لغة الآلة حتى يتم تنفيذها ، حيث لكل لغة مترجماً خاصاً compilerاو مفسر interpreter يقوم بتحويل برامجها للغة الآلة.



الفرق بين المترجم compiler والمفسر interpreter

لنبسط الفرق بينهما
بالجدول التالي

Compiler	Interpreter
يقوم بتحليل البرنامج كامل ثم ترجمته كاملاً	يقوم بتحليل البرنامج سطرا سطر من الكود المصدري
التنفيذ سريع جداً	أبطئ بالتنفيذ
يكتشف الأخطاء بعد تحليل البرنامج كامل	يسهل تتبع الأخطاء لأنه يتوقف عند الخطأ مباشرة

أمثله للغات تستخدم

Compiler	Interpreter
C	Python
C++	PHP
C#	Perl
	Ruby

خطوات تنفيذ البرنامج

1)**المحرر (Editor) :**

هو أداة تستخدم لكتابة البرنامج قد تكون :

مثل: برنامج anaconda وبرنامج visual studio code

2)**يتم تنفيذ البرامج وتشغيلها بإحدى الطريقتين :**

Compiler المترجمات

Interpreter المفسرات

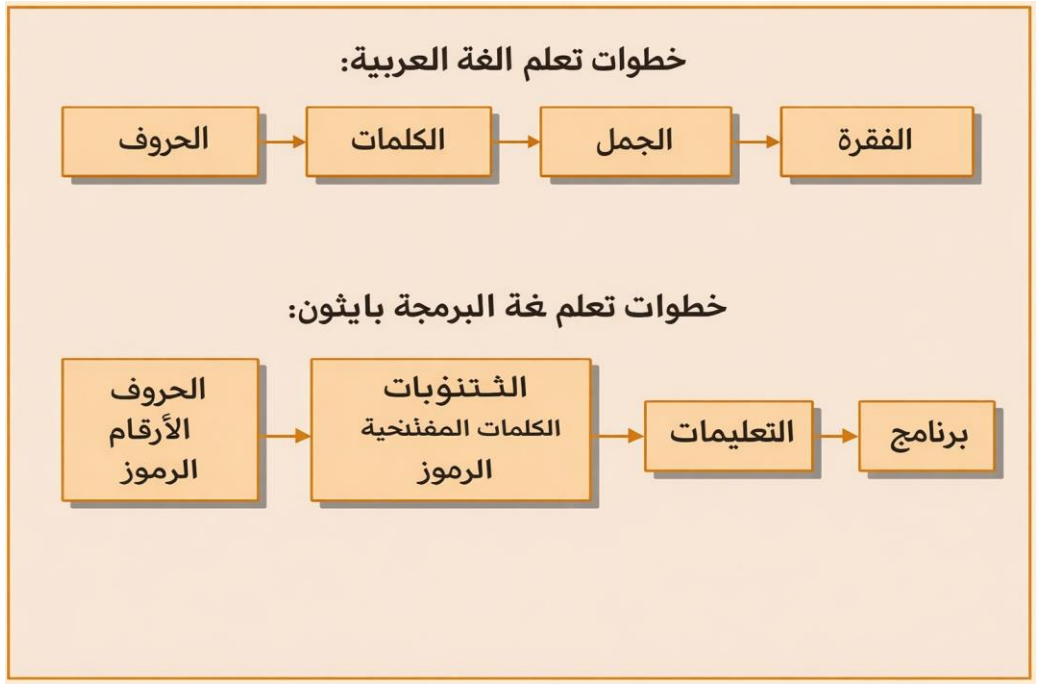
3)**أنظمة التحكم في الإصدارات:** تُستخدم أنظمة التحكم في الإصدارات لتتبع التغييرات التي تطرأ على شفرة المصدر مع مرور الوقت. يتيح ذلك للمبرمجين الرجوع إلى إصدارات سابقة من الشفرة عند الضرورة، والتعاون مع مبرمجين آخرين في نفس المشروع.(GitHub)

4) **أدوات تصحيح الأخطاء:** تساعد أدوات تصحيح الأخطاء المبرمجين على اكتشاف الثغرات والأخطاء في شفرتهم وإصلاحها.

ماهى لغة البايثون؟

- هي لغة برمجة عالية المستوى إبتكرها Guido Van Rossum أثناء عمله في مركز أبحاث Centrum Wiskunde & Informaticaعام 1986 وفى عام 1991 تم نشر أول إصدار منها لتصبح في متناول الجميع
- إذا أنشأت برنامجاً باستخدام لغة بايثون فإنه يعمل على أي نظام في العالم و هذا من أهم ما يدفعك لتعلمها.
- مصدر مفتوح و مجانية
- كائينة التوجه حيث انها تدعم مفهوم الكلاس، الكائن، التغليف، الوراثة إلخ..

البداية مع بايثون



- **الكلمات المفتاحية Keywords** هي الكلمات التي تم تعريف معناها مسبقًا لمترجم بايثون
- بايثون لديها **عدد أكبر من الكلمات المفتاحية** مقارنة بـ ++C، على سبيل المثال Python 3.11 تحتوي على **36** كلمة مفتاحية أساسية مثل: if, else, while, for, def, class وغيرها.
- **الثابت Constant** هو قيمة لا تتغير أثناء تنفيذ البرنامج.
- **المتغير Variable** هو موقع في الذاكرة يمكن أن يخزن قيمة قابلة للتغيير.

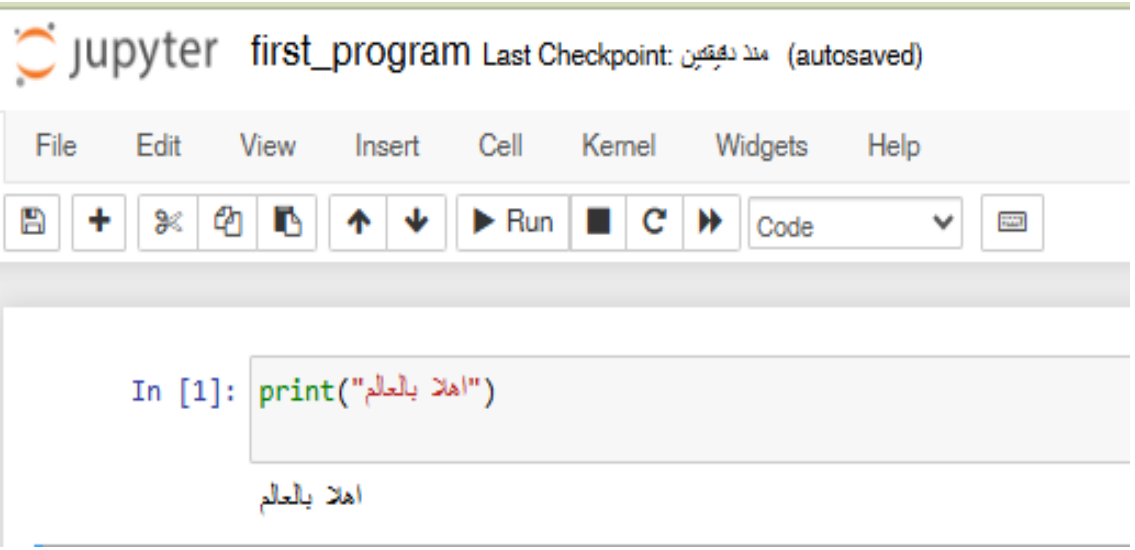
قواعد كتابة البرنامج في بايثون:

- كل تعليمة في البرنامج تُكتب كسطر منفصل :يجب أن تظهر التعليمة بالترتيب الذي نريد تنفيذها به.
- يمكن إدراج مسافات أو فراغات لتحسين وضوح الكود.

يجب مراعاة المسافات البادئة لتحديد البنية.

- بايثون لا تستخدم الفاصلة المنقوطة (;) لإنهاء التعليمة.

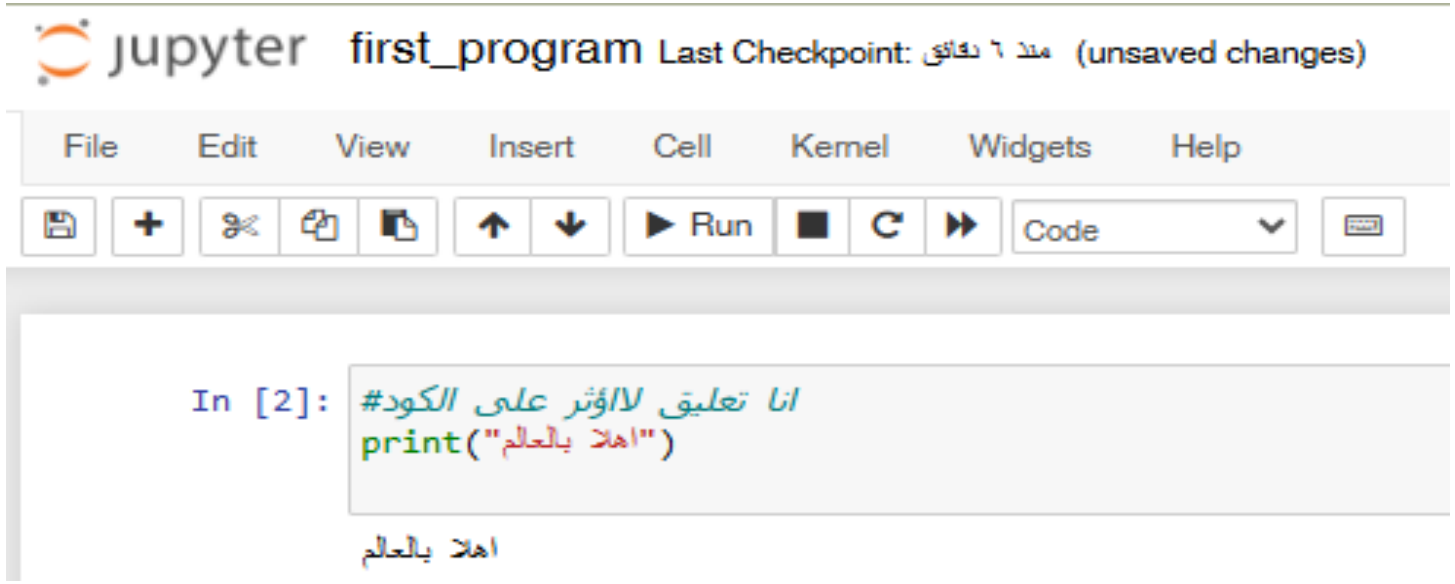
كيف تكتب أول برنامج لك في بايثون :



- سننشئ ملفًا جديدًا باسم
- first_program.ipynb
- بعض الدوال، مثل ()print، هي دوال مُدمجة (built-in) في بايثون بشكل افتراضي.
- هذه الدوال المدمجة متوفرة لنا دائمًا لاستخدامها في برامجنا
- داخل قوسي الدالة ()print توجد سلسلة من الأحرف - اهلا بالعالم! - محاطة بعلامتي اقتباس.
- وهذه الأحرف الموضوعة بين علامات الاقتباس تُسمى سلاسل نصية (strings)

التعليقات

- نستخدم التعليقات لنضع ملاحظات حول الكود الذي كتبناه فقط، لكي لا ننسى كيف برمجنا الكود في حال أردنا مراجعته أو التعديل عليه في وقت لاحق.
- و بالطبع التعليقات لا تؤثر إطلاقاً على الكود المكتوب.
- ويسبق التعليق الرمز #



مفهوم المتغيرات:

- يمكننا **تخزين القيم في ذاكرة الكمبيوتر**.
  كل **موقع تخزين** في ذاكرة الكمبيوتر يسمى **متغير** .**(Variable)**
- **قواعد تسمية المتغيرات في بايثون:**
- يمكن أن يكون اسم المتغير أي **حرف من 1 إلى 31 حرفًا** أو رقمًا أو **(_) underscore**
- يجب أن يكون **الحرف الأول** في اسم المتغير **حرفًا** أو **(_) underscore**
- لا يُسمح بوجود **مسافات أو فواصل** داخل اسم المتغير
- لا يمكن استخدام أي **رمز خاص** آخر غير (_) underscore

معايير تسمية المتغيرات

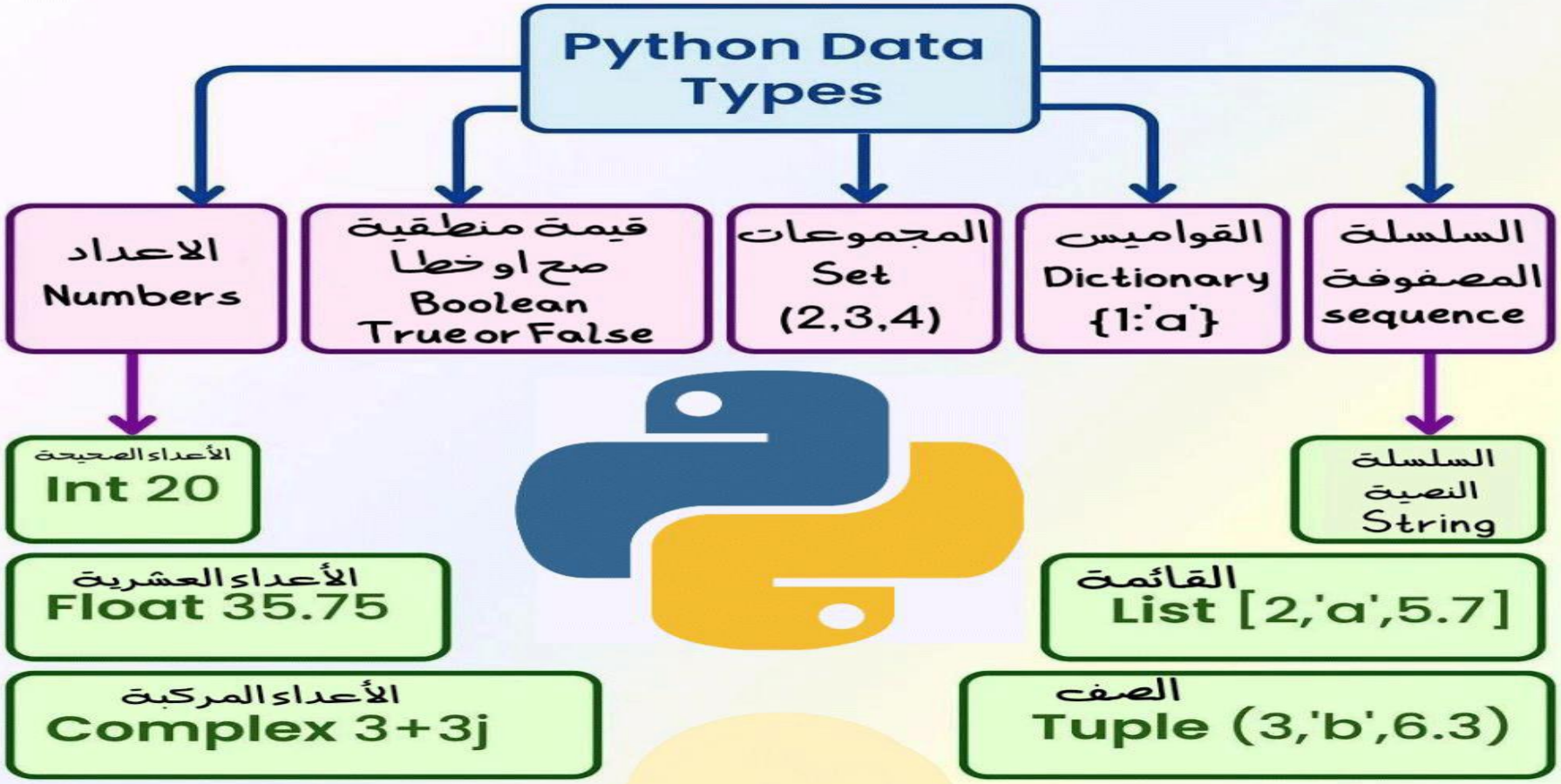
- **ابدأ اسم المتغير بحروف صغيرة**
- **استخدم أسماء واضحة ومعبرة** عن محتوى المتغير
- **لفصل الكلمات داخل اسم المتغير يمكن استخدام:**
- **(mixedCase)** الحروف الكبيرة والصغيرة

حاول ان تحل

- **ايا من ما ياتى تسميه صحيحه وايها خاطئة:**

salary=20	✓
2salary=30	✗
‘salary=30	✗
_salary=20	✓
!salary=20	✗

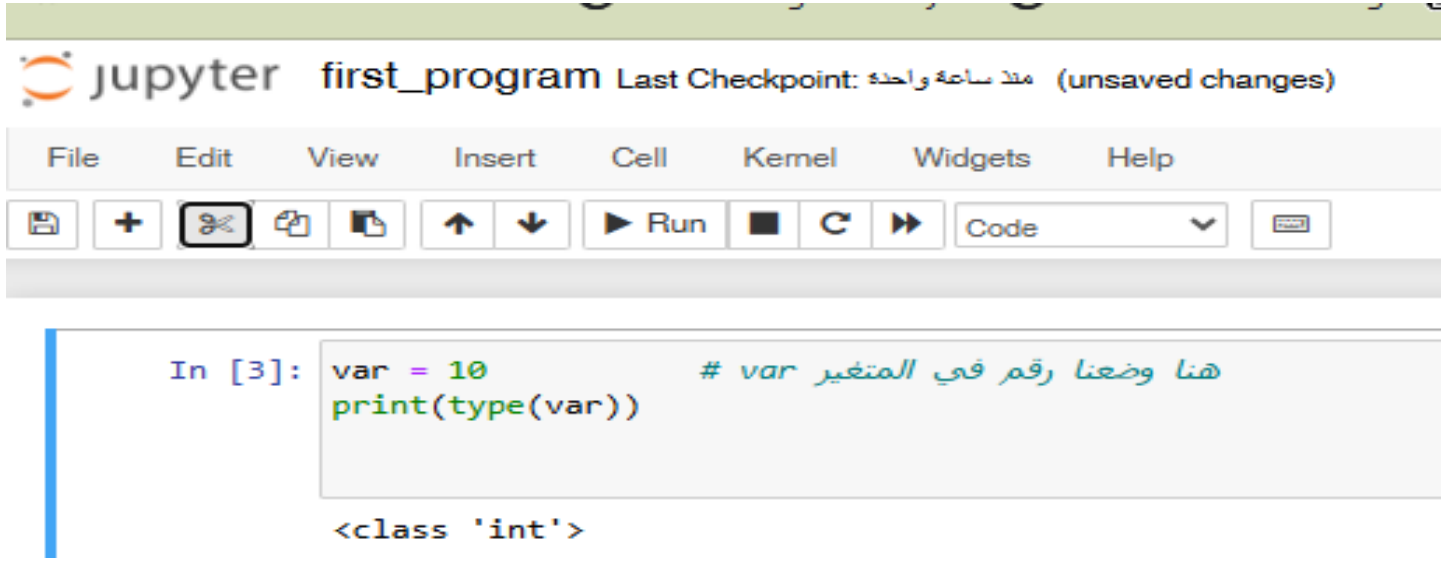
أنواع البيانات في بايثون باختصار



النوع	إستخدامه
Int رقم صحيح	يستخدم لتخزين أعداد صحيحة. مثال x = 3
Float رقم عشرى	يستخدم لتخزين أعداد تحتوي على فاصلة عشرية. مثال x = 1.5
Complex اعداد مركبة	يستخدم لتخزين أعداد مركبة (Complex Number) و التي غالباً ما يحتاجها المهندسون عند إجراء عمليات حسابية معقدة. و هنا يجب وضع الحرف J أو j بعد العدد حتى يعلم مفسّر بايثون أننا نقصد عدد مركب و ليس عدد عادي. مثال x = 4J
السلاسل النصيه string	النصية (string) هي عبارة عن تسلسل من محرف واحد أو أكثر (محارف وأعداد ورموز)، ويمكن أن تكون ثابتة أو متغيرة X="hello"
القيم المنطقيه bool	عند إسناد القيمة True أو القيمة False إلى المتغير فإنه يصبح من النوع bool. x=true

معرفة نوع المتغير

- لمعرفة نوع أي متغير يمكنك إستخدام الدالة **type()**.



ما نوع المتغيرفيما ياتى

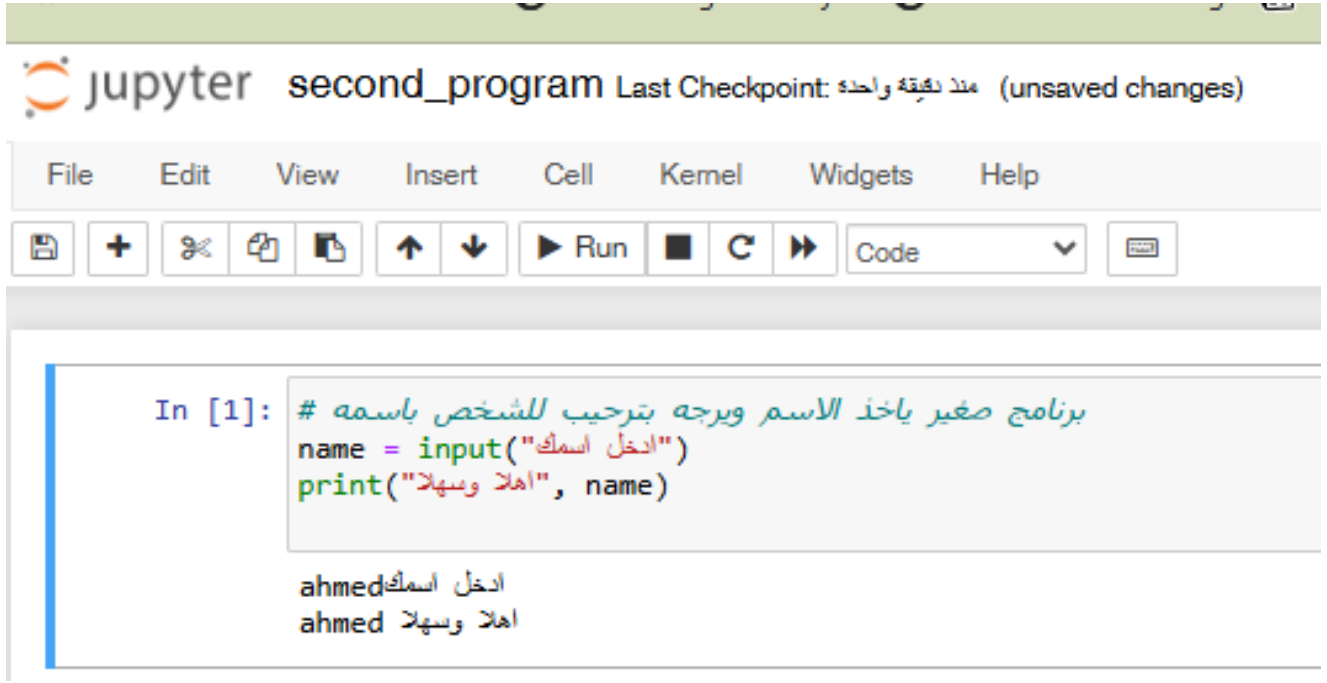
x= 100	Int
salary=350.25	Float
age = 30	Int
name = “Ahmed”	String
is_student = False	bool

ما هو input()؟

- دالة مدمجة في بايثون تُستخدم لقراءة بيانات من المستخدم أثناء تشغيل البرنامج.

البرنامج يتوقف وينتظر إدخال المستخدم
القيمة المدخلة تُخزن في متغير

- ملاحظة هامه :
- الدالة input دائما ترجع نوع string



العمليات الحسابيه فى بايثون

- كما هو الحال مع أي لغة برمجة أخرى، يمكن استخدام عمليات الجمع والطرح والضرب والقسمة مع الأرقام.

```
print(1 + 2 * 3 / 4.0)
```

- عامل آخر متاح هو عامل المودولوس (%)، الذي يعيد باقي القسمة الصحيح. المقسوم % المقسوم عليه = الباقي.

```
print(11 % 3)
```

ترتيب العمليات الحسابية من الأعلى للأدنى

- في لغة Python يتم تنفيذ العمليات الحسابية بالترتيب التالي:
- 1 **الأقواس ( )**
  2 **الأسس ****
  3 **الضرب والقسمة * / // %**
  4 **الجمع والطرح + -**

عند تساوي العمليات في نفس المستوى يتم التنفيذ من **اليسار إلى اليمين**

مثال

```
print(5 + 3 * 2)
```

- الحل:
- 3 × 2 = 6
- 5 + 6 = 11

الناتج = **11**

مثال

```
print(20 / 5 * 2)
```

- الحل:
- 20 ÷ 5 = 4
- 4 × 2 = 8
- الناتج = **8**

مثال

```
print(10 - 4 + 3)
```

- الحل:
- 10 - 4 = 6
- 6 + 3 = 9
- الناتج = **9**